

Ideia do padrão decorator em um sistema de hamburgueria

Raul Beltrame¹, Matheus Dias¹

¹Curso de Programação Orientada a Objetos
Sistemas de Informações - UFVJM

{raul.beltrame,matheus.dias}@ufvjm.edu.br

Resumo. Neste artigo analisamos qual padrão de projeto era o mais simples e coerente para aplicarmos em um sistema de hamburgueria desenvolvido em Java na disciplina programação orientada a objetos. Neste estudo comparamos padrões candidatos com a estrutura atual considerando esforço de implementação, impacto nas classes que já temos e também se tinha uma certa clareza didática. Nos estudos pensamos que o padrão Decorator era a alternativa mais adequada para o projeto, pois o sistema já possui produtos, adicionais e itens de pedido que preço final depende de opcionais.

Abstract. In this paper, we analyze which design pattern was the most straightforward and consistent to apply to a burger shop system developed in Java for an object-oriented programming course. We compare candidate patterns against the current structure, considering implementation effort, impact on existing classes, and pedagogical clarity. Our study indicates that the Decorator pattern is the most appropriate alternative for the project, as the system already features products, add-ons, and order items whose final price depends on optional components.

1. Introdução

Os Padrões de projeto são soluções reutilizáveis para problemas recorrentes de desenvolvimento de software orientado a objetos. Eles não representam exatamente código pronto, mas formas de colaboração entre classes que ajudam a organizar responsabilidades, reduzir acoplamento e melhorar a manutenção do sistema gamma1994. Essa preocupação também se relaciona com critérios de qualidade de software que venho estudando.

Kleppmann destaca confiabilidade, escalabilidade e manutenibilidade como objetivos importantes na construção de sistemas, além de associar a manutenibilidade a qualidades como simplicidade, operabilidade e facilidade de evolução. Mesmo em um projeto de menor escala, esses conceitos ajudam a justificar o uso de padrões de projeto, pois uma solução bem organizada tende a ser mais fácil de entender, reusar, alterar e até para nossa apresentação.

Falando do projeto, o projeto analisado é um sistema de hamburgueria desenvolvido em Java. Ele possui classes para clientes, funcionários, produtos, pedidos, vendas, estoque, entregas e persistência em JSON. Dentro desse escopo, uma regra importante é a montagem de um pedido a partir de um produto base e de adicionais escolhidos pelo cliente. Essa característica aproxima o sistema do padrão Decorator, cujo objetivo é acrescentar responsabilidades ou comportamentos a um objeto de forma flexível, sem criar uma grande hierarquia de subclasses.

Entre os padrões listados como opções, também consideramos o Singleton. Porém, considerando o contexto da hamburgueria, achamos que o Decorator apresenta a melhor relação entre facilidade de implementação, utilidade no domínio e clareza para explicação acadêmica. Além disso, ele nos pareceu mais desafiador do que o Singleton.

2. Objetivo

O objetivo deste trabalho é identificar o padrão de projeto mais adequado para ser implementado no sistema de hamburgueria, alinhando a escolha com base na estrutura atual do código. Especificamente, buscamos avaliar o uso do padrão Decorator na representação de produtos que já eram personalizados/decorados com adicionais, demonstrando como ele pode melhorar a organização da regra de cálculo do subtotal de um item de pedido.

3. Material e Métodos

O material que trabalhamos em cima foi o código do sistema de hamburgueria, estruturado como um projeto em Java. A análise concentrou-se nas nossas classes `Produto`, `DescricaoProduto`, `Adicional`, `ItemPedido`, `Pedido`, `CadastroPedidos`, `GeradorId` e `PersistenciaJSON`.

O método adotado foi uma análise qualitativa do código. Para cada padrão candidato, foram considerados quatro ideias:

- Esforço para implementar.
- Quantidade de classes a mudar.
- Coerência com as regras de negócio.
- Facilidade de explicação no artigo e na apresentação.

4. Resultados e Discussão

Nosso estudo mostrou que o padrão Singleton seria o mais curto de implementar, pois a classe `GeradorId` já centraliza a geração de identificadores por meio de atributos e métodos estáticos. Bastaria transformar essa classe em uma instância única controlada. Porém, essa solução teria impacto funcional limitado, pois apenas formalizaria um comportamento que já está centralizado no projeto.

O padrão Decorator, por outro lado, encaixa diretamente na regra de negócio dos pedidos. Atualmente, a classe `ItemPedido` possui um `Produto` e uma lista de objetos `Adicional`. O método de cálculo do subtotal soma o preço base do produto com os valores dos adicionais e multiplica o resultado pela quantidade. Essa estrutura já representa, de forma simplificada, a ideia central do Decorator que é a partir de um objeto base e acrescentar funcionalidades ou valores sem modificar diretamente a classe original.

Uma implementação formal poderia criar uma interface, por exemplo `ItemCardapio`, com métodos para obter descrição e valor. O produto base implementaria essa interface, enquanto cada adicional seria representado por um decorador que também implementa a interface e mantém uma referência para outro `ItemCardapio`. Assim, um hambúrguer poderia ser decorado com bacon, queijo, molho extra ou outros adicionais de forma encadeada.

A Tabela 1 apresenta uma síntese comparativa dos padrões avaliados para o sistema.

Tabela 1. Comparação dos padrões mais viáveis para o projeto

Padrão	Aplicação no projeto	Avaliação
Singleton	Geração única de identificadores na classe <code>GeradorId</code> .	Muito simples, mas pouco expressivo.
Decorator	Personalização de produtos com adicionais em <code>ItemPedido</code> .	Simples e muito aderente ao domínio.
Factory Method/Builder	Criação de pedidos e produtos complexos.	Possível, mas menos direto que Decorator.

Com base nessa comparação, o Decorator foi considerado por nós o padrão mais adequado para o trabalho. Ele não exige mudanças profundas na arquitetura, aproveita classes já existentes e melhora a separação de responsabilidades. A classe `Produto` pode continuar representando o item cadastrado no cardápio, enquanto os adicionais passam a ser tratados como elementos que decoram o item base durante o cálculo do preço e da descrição final[.

Outro ponto positivo é a clareza didática. Em uma hamburgueria, é natural explicar que um lanche base recebe complementos opcionais. Isso facilita a apresentação do padrão e mostra uma aplicação prática, diferente de uma implementação meramente artificial. Os demais padrões da lista também poderiam ser aplicados em partes específicas do sistema. Porém, para este projeto, eles exigiriam a criação de regras novas ou uma remodelagem maior do código, o que achamos menos indicados para uma implementação simples e objetiva.

5. Conclusão

Concluimos que o padrão Decorator é a melhor escolha para o contexto do sistema de hamburgueria. Embora o Singleton seja tecnicamente o padrão com menor alteração, o Decorator apresenta melhor aderência ao escopo, pois representa diretamente a personalização de produtos com adicionais, como já abstraímos.

A implementação proposta permite calcular o valor de um item de pedido de forma mais flexível e organizada, sem criar subclasses para cada combinação de hambúrguer e adicional. Dessa forma, o padrão contribui tanto para a qualidade do código quanto para a clareza da explicação acadêmica do projeto.

Referências Bibliográficas

- [1] Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [2] Freeman, E., Freeman, E., Sierra, K. and Bates, B. (2004). *Head First Design Patterns*. O'Reilly Media.
- [3] Kleppmann, M. (2017). *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media.

- [4] Sociedade Brasileira de Computação (2017). *SBC Conferences Template – updated sbc-template.sty v2017*. Disponível em: <https://www.overleaf.com/latex/templates/sbc-conferences-template-updated-sbc-template-dot-sty-v2017/pyhttxftxjqn>. Acesso em: 05 maio 2026. [cite: 48]